

Decoding Neural Networks: Analyzing Alignment Between Mid-Level Vision Feature Detection and Human Perception

Motivation

- Humans rely on symmetry, orientation, and curvature for visual perception (mid-level vision).
- We analyze how neural networks detect these features by extracting feature maps.
- Focus: Assess how well neural networks encode **orientation**.
- Purpose: Compare neural network feature detection to human perception.

Background Information

Convolutional Neural Networks (CNNs)

- Hierarchical neural networks designed for automatic feature extraction and spatial hierarchy learning, primarily used for image recognition and processing.
- Early layers detect edges and orientations; deeper layers capture shapes and objects.

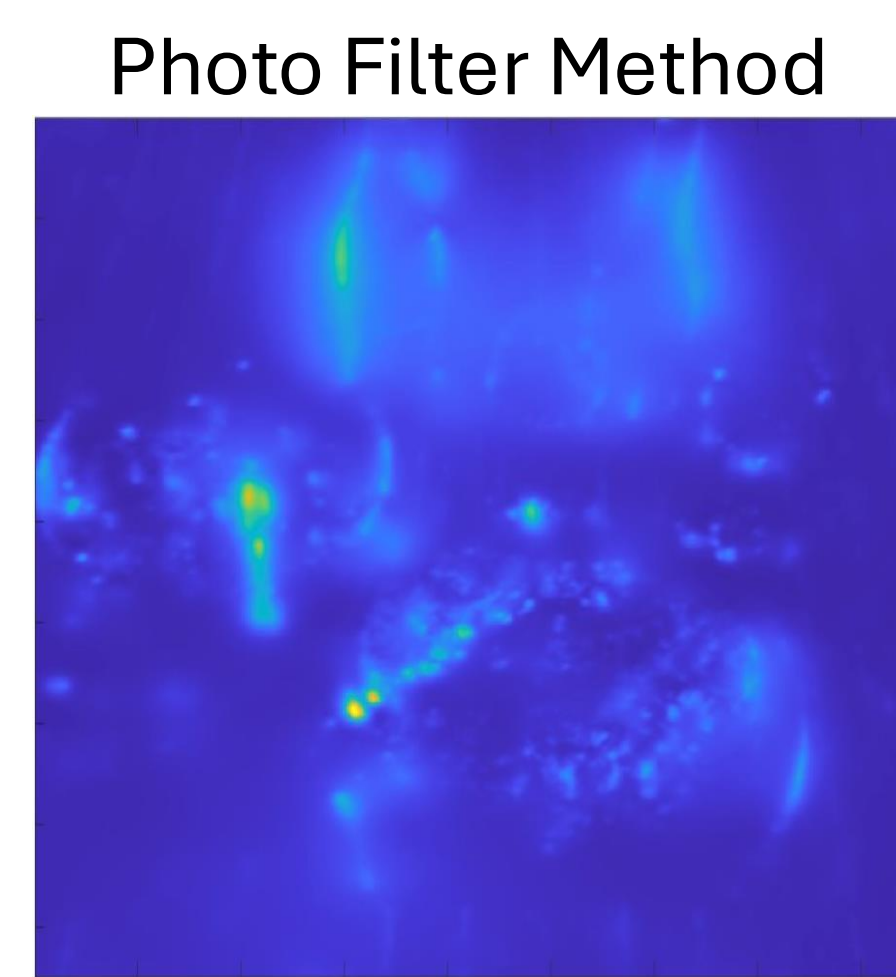
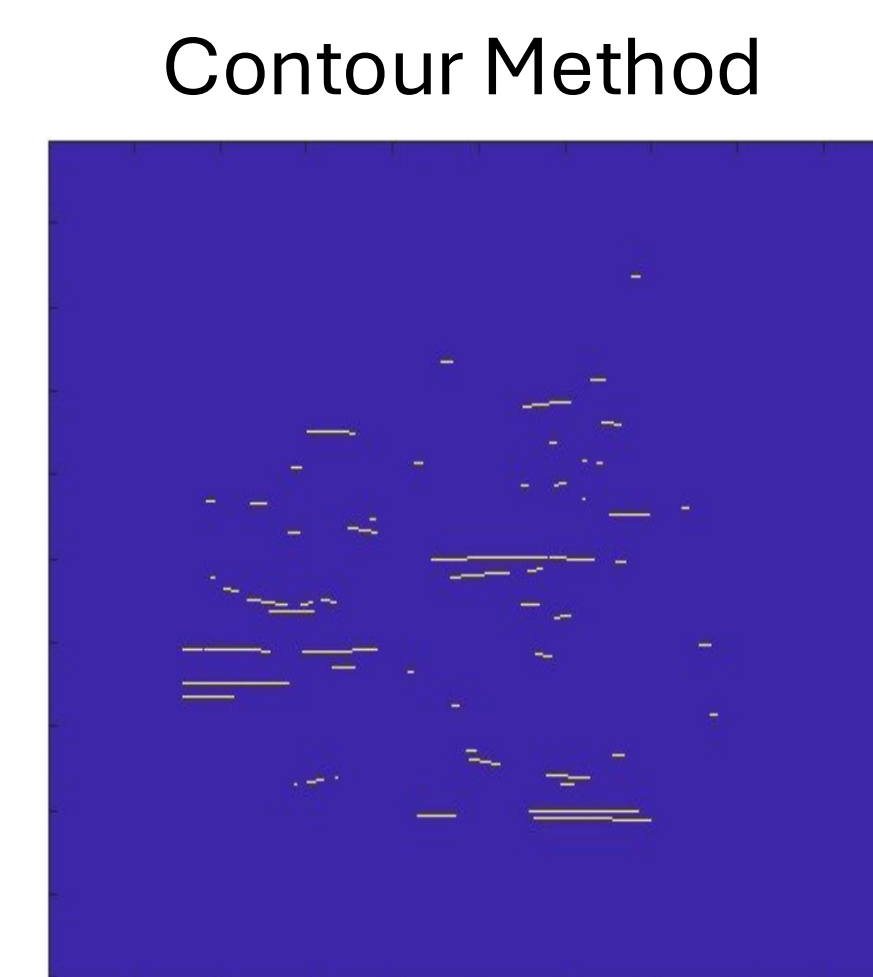
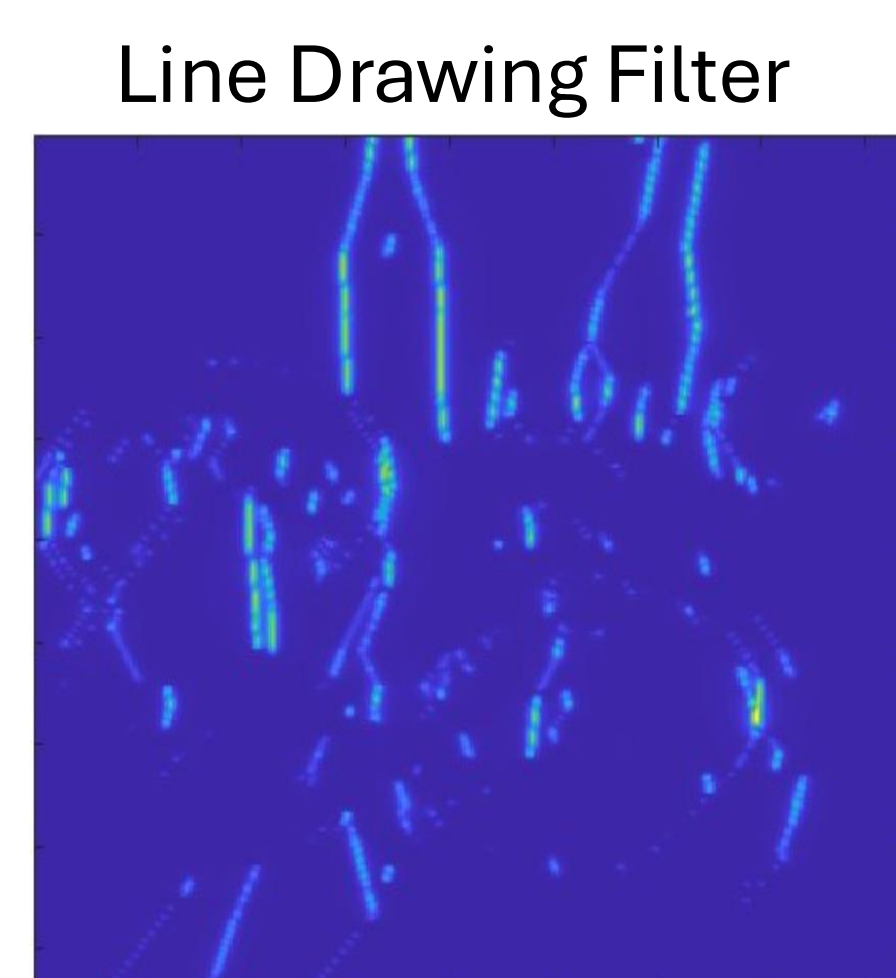
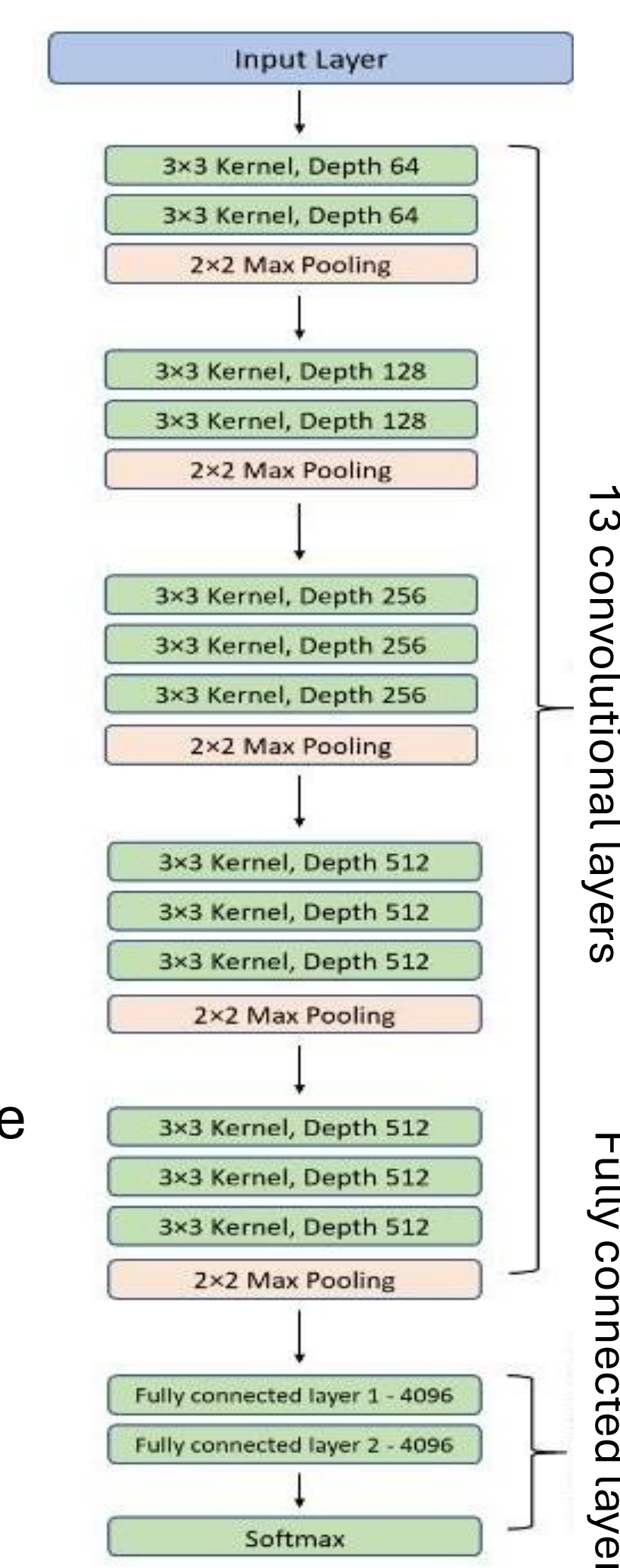
VGG16 Architecture

- A deep CNN with 16 layers
- Study looks into the first layer, with 64 feature maps

Feature Maps & Orientation Detection

- Studies have shown VGG16's early feature maps reveal how the network processes orientation.
- We analyze these maps to compare neural network vision with human perception.

Filter Methods



Research Question

"Do different filter methods reveal whether certain feature maps in the first layer of VGG16 are better at detecting specific edge orientations?"

Methodology

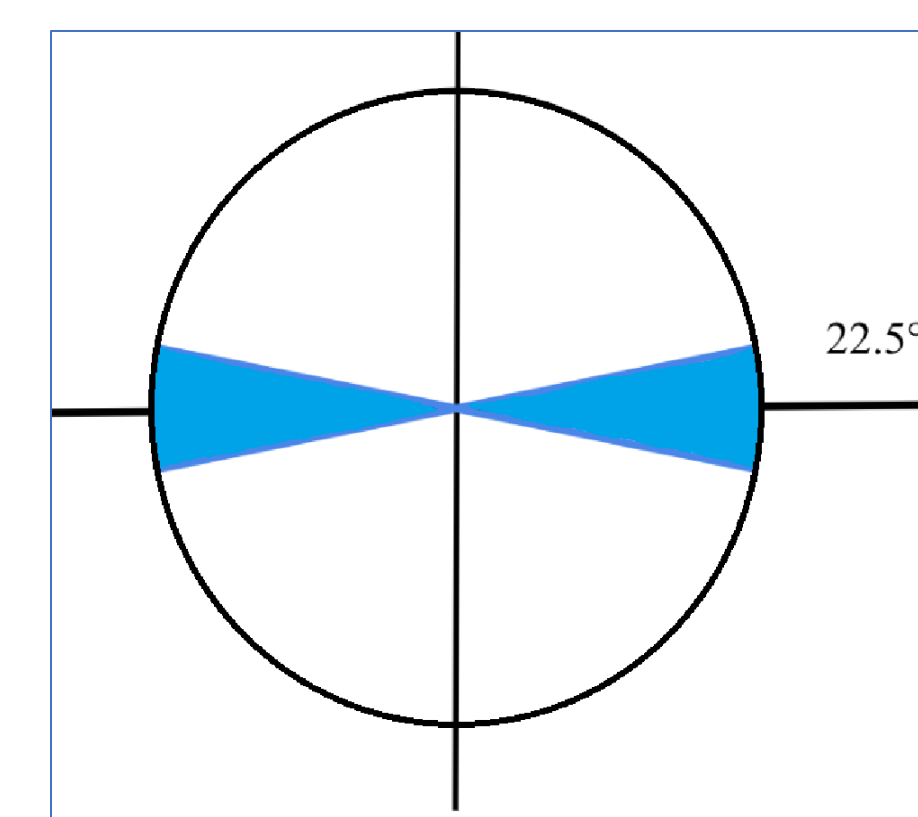
We will be using 14600 images from the ImageNet dataset.

The process below will be repeated 3 times for:

- Line Drawing Filter Method
- Photo Filter Method
- Contour Method

Step 1: Compute Edge Orientation (Baseline)

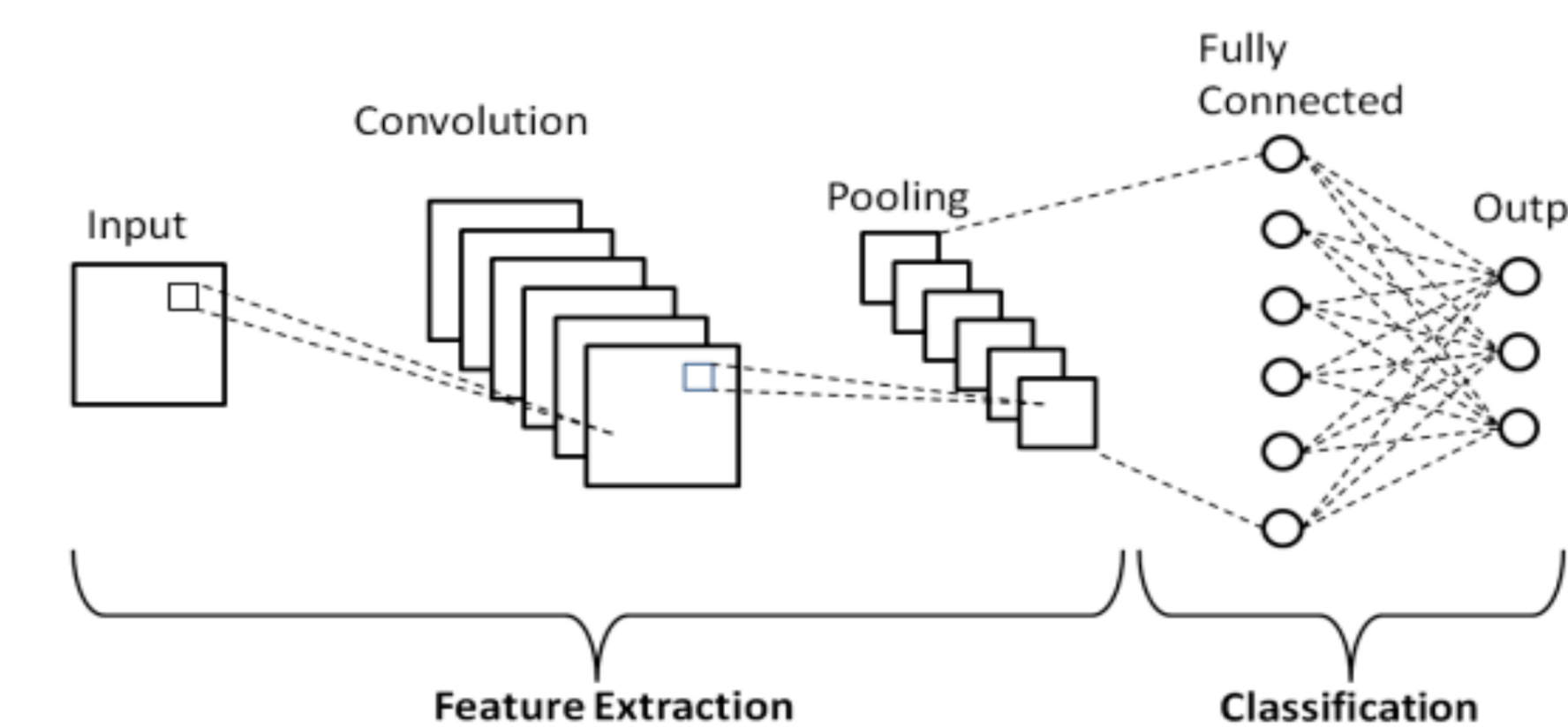
- Apply the edge detection methods on each of the 14600 images.
- Extract edges and classify their orientations into 8 bins (each covering 22.5°).
- Generate Oribin Maps (binary matrices) to represent edge presence in each orientation.



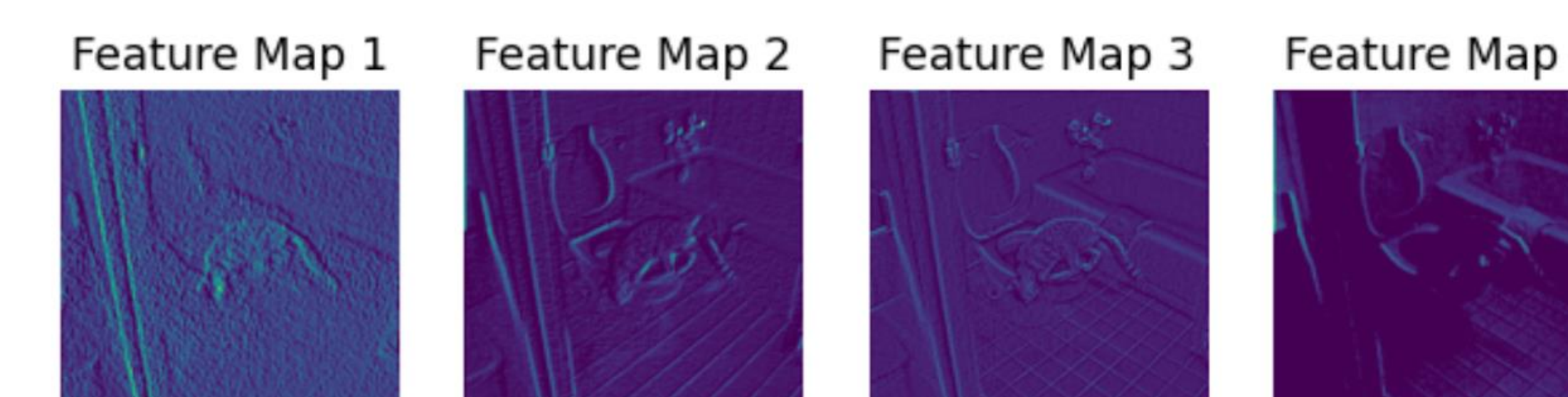
oribinmap{1, 1}					
	01	02	03	04	05
302	0	0	0	0	0
303	0	0	0	0	0
304	1	1	1	1	1
305	0	0	0	0	0
306	0	0	0	0	0
307	0	0	0	0	0
308	0	0	0	0	0
309	0	0	0	0	0
310	0	0	0	0	0
311	0	0	0	0	0

Step 2: Extract Feature Maps from VGG16

- Pass the original image through the first convolutional layer of VGG16.
- Extract 64 feature maps, each representing detected features at different locations.
- Convert feature maps into intensity matrices for visualization and analysis.



Note: feature maps can be visualized by converting them into images



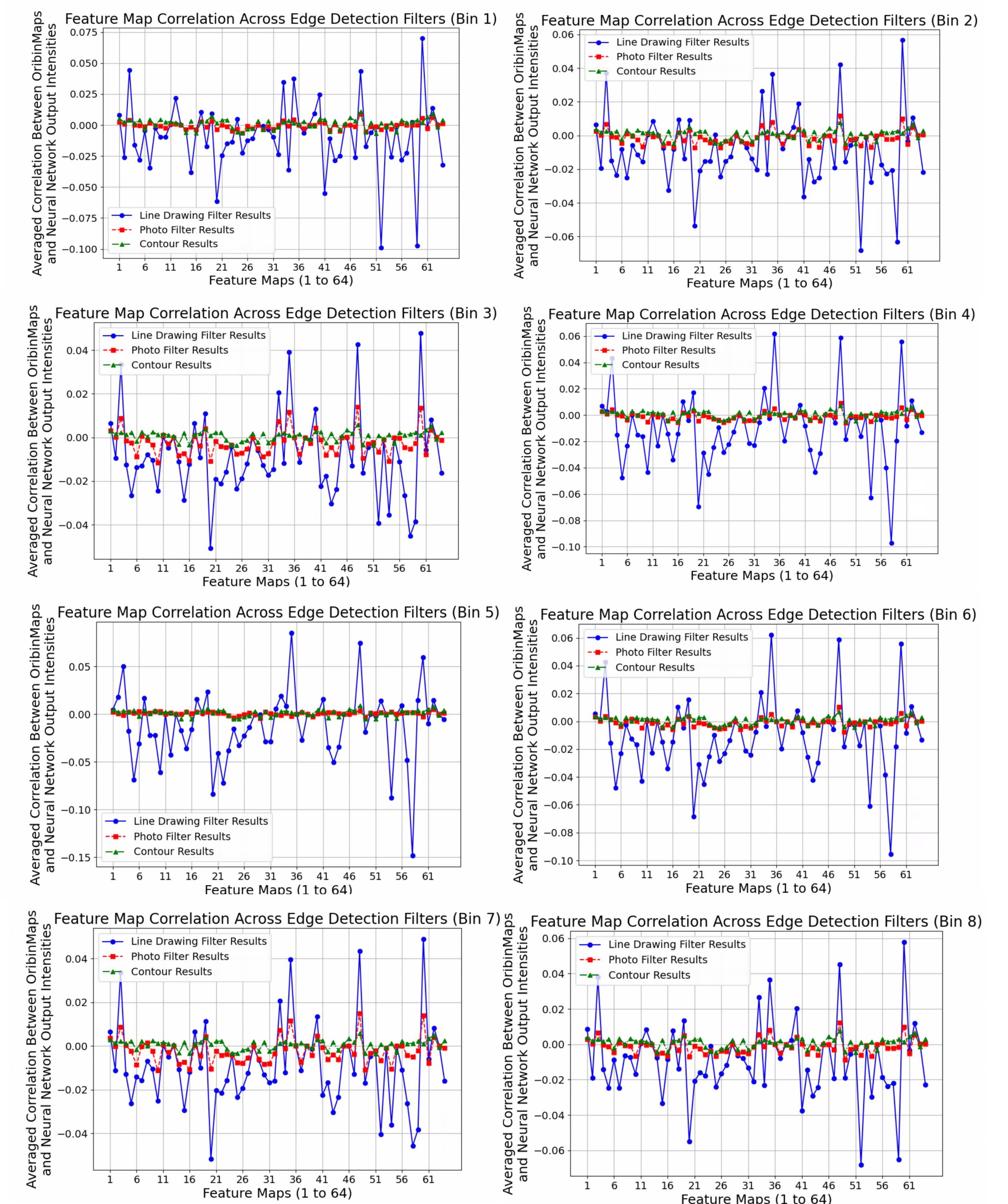
Step 3: Compare Feature Maps with Oribin Maps

- Extract intensity matrix (2D) from each of the 64 feature maps and compare the intensity matrices of the feature maps with Oribin Maps.

$$\rho_{XY} = \frac{\text{Covariance}(X, Y)}{\sigma_X \sigma_Y}$$

- Identify which feature maps best capture edges of specific orientations.
- Repeat for all 8 bins to evaluate VGG16's ability to detect edge orientations.

Results



Key Findings and Interpretation

- Peaks and Dips in intensity for each bin
- Despite consistent patterns, we see intensity scaling differences across filter methods
- All three filter methods show similar patterns, or structural consistency, although absolute intensity values vary.

Conclusion

- Different filter methods reveal that certain feature maps in VGG16 are better at detecting specific edge orientations.
- Feature maps work together with each contributing uniquely to capturing different edge orientations